SPÉCIFICATION

SPÉCIFICATION (SPEC) — le langage commun du SDLC moderne

> **Note terminologique.** Cette page explique en profondeur le concept de **spécification (specification / spec)** dans le SDLC moderne. Pour la définition courte et officielle du terme, utiliser le Lexique / Terminologies comme source de référence. Cette page ne remplace pas le lexique : elle explique le pourquoi, l'origine, les impacts et les usages opérationnels.

Idée centrale

Une spécification est le langage commun qui décrit le produit attendu, ses comportements, ses dépendances, ses contraintes, ses environnements, ses configurations, ses signaux et ses preuves observables.

Dans le SDLC actuel, la spécification n'est plus seulement un document rédigé avant le développement. Elle peut être dans le code, dans un outil de validation, dans un contrat d'interface, dans une règle métier, dans une configuration, dans un modèle, dans une requête, dans un diagramme ou dans une preuve automatisée.

La règle à retenir est simple : **si cela décrit précisément un comportement attendu sous condition, c'est une spécification.**

La forme universelle : IF... THEN...

Une spécification est une forme de condition. Elle commence toujours par **IF** et contient toujours un **THEN**.

> **IF** une condition, un contexte, une entrée, un état, une configuration, un événement ou une dépendance est vrai, **THEN** un comportement, un résultat, une contrainte, un effet, une donnée, un signal ou une preuve doit être observable.

Cette forme est volontairement simple. Elle traverse les outils, les métiers et les niveaux de détail. Elle est lisible par l'humain, exploitable par la machine, compatible avec l'automatisation et beaucoup plus efficace pour l'intelligence artificielle, parce qu'elle donne des instructions précises.

On ne cherche donc pas à maintenir plusieurs langages parallèles pour dire la même chose : critères d'acceptation, conditions de succès, exigences, cas de test, scénarios, Gherkin, BDD, AAA ou « As a / I want / So that ». Ces formes peuvent avoir existé historiquement, mais elles convergent maintenant vers une nomenclature commune : la spécification IF… THEN….

> **Conseil pratique.** Il n'y a pas de grand secret : on devient forgeron en forgeant. Utiliser le format **IF… THEN…** et catégoriser les spécifications aide toutefois grandement à rendre l'exercice concret, partageable et automatisable.

Pourquoi ce changement est nécessaire

Historiquement, chaque domaine a développé son propre vocabulaire : le portfolio parlait de besoins et d'exigences, le produit de stories et de critères d'acceptation, le développement de tests unitaires, QA de cas de test et de scénarios, les opérations de signaux et d'incidents, l'architecture de contrats et de modèles.

Le résultat était souvent une multiplication de couches pour décrire le même produit : une exigence, puis une story, puis des critères, puis des cas, puis des scénarios, puis des tests, puis des preuves. Cette superposition ajoute de la complexité, crée des écarts de compréhension et rend l'automatisation plus difficile.

Le concept moderne de spécification brise ces barrières. Il fournit une nomenclature unique pour définir le produit complet et ses dépendances, sans forcer chaque discipline à traduire le même sens dans son propre langage.

Un test est-il une spécification ? Oui.

Quand un développeur demande : « Est-ce que mes 2000 tests unitaires sont aussi des spécifications ? », la réponse est toujours : **oui**, s'ils décrivent des comportements attendus de façon précise et vérifiable.

Un test unitaire dans le code, un test dans Xray, une validation Playwright, une règle métier, un contrat API, une assertion, une requête, une configuration attendue ou une preuve de non-régression peuvent tous être des spécifications.

Le point important n'est pas l'outil ni l'endroit où la définition est stockée. Le point important est la fonction : **décrire précisément ce qui doit être vrai.**

Les anciens termes convergent vers la spécification

Ancien langage	Convergence moderne

Exigence	Devient une spécification lorsqu'elle est formulée comme condition vérifiable IF... THEN....
Critère d'acceptation	Devient une spécification utilisée pour décider si le comportement attendu est satisfait.
Condition de succès	Devient une spécification qui décrit le résultat observable attendu.
Cas de test	Devient une spécification ou un jeu de données associé à une spécification.
Scénario	Devient une suite de spécifications ou un contexte d'exécution.
Test	Devient une spécification exécutable lorsque le comportement attendu est explicite.
Gherkin / BDD	Peut inspirer la discussion, mais la source de vérité cible demeure IF... THEN....
AAA	Peut structurer le code, mais la définition attendue demeure IF... THEN....
As a / I want / So that	Peut exprimer l'intention, mais ne remplace pas la condition vérifiable.

Guide pratique de rédaction

Une façon simple de produire des spécifications utiles est de travailler en petites passes collectives :

1. Présenter et expliquer le haut niveau des spécifications à produire.
2. Décrire d'abord la fonctionnalité avec les différents angles de vue des participants : produit, développement, validation, support, opérations, client.
3. Comparer les visions, repérer les différences, les ambiguïtés et les hypothèses, puis en discuter.
4. Transformer les éléments clarifiés en spécifications formulées en **IF... THEN...**.
5. Faire une première passe de contrôle : a-t-on tout ce qu'il faut pour implémenter, automatiser, valider et observer à tous les niveaux pertinents ?

Spécification-driven : SDD, ATDD, FDD, DDD, BDD, TDD, data-driven

La spécification devient le point de rassemblement de plusieurs approches : SDD, ATDD, FDD, DDD, BDD, TDD, data-driven et autres pratiques orientées comportement, domaine, données ou automatisation.

L'objectif n'est pas d'empiler ces pratiques comme des méthodes concurrentes. L'objectif est de reconnaître qu'elles cherchent toutes à mieux définir ce qui doit être vrai avant, pendant et après l'implémentation.

Dans un SDLC moderne, tout devient progressivement **driven par les spécifications** : la clarification, la planification, l'implémentation, l'intégration, la validation, la diffusion, l'observabilité, la confirmation terrain et l'amélioration continue.

Le rôle critique de SPÉCIFIER dans le SDLC

SPÉCIFIER relie ÉLICITER et DÉFINIR. C'est l'endroit où les besoins, signaux, incidents, contraintes, risques et intentions sont transformés en définitions précises, testables, traçables et exploitables.

Plus les spécifications sont précises, plus le reste du SDLC peut être efficace. Une spécification claire réduit les ambiguïtés avant la planification, guide l'implémentation, rend l'automatisation possible, facilite les preuves de conformité et améliore la qualité des décisions Go / No-Go.

À l'inverse, si on spécifie mal, on transfère l'ambiguïté vers le code, les tests, les opérations, le support et les clients. L'IA amplifie ce phénomène : elle répond mieux lorsque les instructions sont précises, mais elle peut amplifier le flou si la définition est vague.

Spécifier le produit complet

Une spécification peut exister à très haut niveau ou à un niveau atomique. Elle peut viser un comportement produit, une règle métier, une interface, une dépendance, un environnement, une configuration, un profil, un seuil de performance, un signal d'observabilité, une stratégie de rollback ou une contrainte de sécurité.

Les spécifications peuvent être classées par niveau, interface, angle ou type : small, medium, large ; UI, API, machine ; black-box, gray-box, white-box ; fonctionnel, performance, sécurité, résilience, observabilité, supportabilité, etc.

Le but n'est pas de créer une nouvelle couche documentaire. Le but est de faire de la spécification le point de vérité qui relie le produit, son code, ses validations, ses environnements, ses configurations, ses dépendances et ses preuves.

Shift-left et shift-right : les spécifications comme pont

Les spécifications permettent de relier naturellement le **shift-left** et le **shift-right**. Le shift-left consiste à clarifier, vérifier et automatiser le plus tôt possible, avant ou pendant l'implémentation. Le shift-right consiste à observer le produit en conditions réelles, après la diffusion, à partir des signaux terrain, des métriques, des incidents et des usages.

La spécification IF… THEN… devient le lien entre ces deux mouvements : elle décrit ce qui devrait être vrai avant l'implémentation, puis elle sert aussi de référence pour confirmer, en production ou en contexte opérationnel, si ce qui devait être vrai l'est réellement.

Moment	Rôle des spécifications
Avant l'implémentation — shift-left	Définir précisément les comportements attendus, les dépendances, les environnements, les configurations, les profils, les contraintes et les preuves attendues. Cela permet d'automatiser tôt, de générer des validations, de guider le code et d'utiliser l'IA avec des instructions précises plutôt qu'avec des souhaits vagues.
Pendant l'implémentation	Relier le code, les tests unitaires, les contrats, les validations, les données et les configurations à la même définition IF… THEN…. Le développement devient guidé par ce qui doit être vrai, et non par des interprétations dispersées.

Après la diffusion — shift-right	Comparer les signaux réels — logs, métriques, traces, incidents, usage, performance, stabilité, disponibilité, supportabilité, ROI — avec les comportements et seuils spécifiés. Les observations terrain alimentent ensuite l'amélioration continue et les prochaines spécifications.

Avec cette approche, on ne sépare plus artificiellement la définition, le test et l'observation. Une spécification peut être utilisée pour générer une validation avant le code, guider l'implémentation, produire une preuve automatisée, puis devenir un signal observable en opération. C'est ce qui rend possible un SDLC réellement piloté par les spécifications.

IA et automatisation : le game changer

L'intelligence artificielle et l'automatisation changent l'importance de la spécification. Pour obtenir des réponses précises, générer du code utile, produire des validations fiables, analyser des écarts ou orchestrer des pipelines, il faut des instructions précises.

La spécification IF… THEN… fournit ce format commun. Elle est assez simple pour être comprise par tous, mais assez structurée pour être exploitée par des machines, des outils d'automatisation, des générateurs de tests, des pipelines CI/CD, des outils de modélisation et des assistants IA.

C'est pourquoi on voit de plus en plus d'outils utiliser le vocabulaire de spec, notamment dans les écosystèmes GitHub, Microsoft, Playwright et plusieurs autres. L'industrie converge vers cette idée parce qu'elle est nécessaire pour travailler efficacement avec l'IA et l'automatisation.

Qualimétrie, signaux et amélioration continue

Les spécifications ne servent pas seulement à construire le produit. Elles servent aussi à observer sa santé.

Chaque spécification peut produire ou relier des signaux : résultat de validation, métrique, log, trace, incident, taux d'échec, temps de réponse, adoption, retour utilisateur, disponibilité, stabilité, supportabilité ou ROI.

La qualimétrie utilise ces signaux pour comprendre la santé du produit, des processus, du cadre de travail, de la livraison, des opérations et de la valeur produite. Les spécifications donnent

donc une structure aux métriques : on ne mesure pas seulement pour mesurer, on mesure pour savoir si ce qui devait être vrai l'est réellement, et pour alimenter l'amélioration continue.

Règle de gouvernance

> ⚠️ **Ne pas recréer plusieurs langages après avoir spécifié.** Si une spécification existe, on ne recrée pas ensuite une exigence séparée, un critère séparé, un scénario séparé, un cas séparé et un test séparé pour dire la même chose. On relie les artefacts nécessaires à la même spécification, ou on rend la spécification exécutable.

La spécification est la définition commune. Les outils peuvent varier, les niveaux peuvent varier, les angles peuvent varier, mais le langage cible reste le même : **IF... THEN...**.

📚 Références

- Lexique / Terminologies — définition officielle des termes et source de vérité terminologique.
- SDLC officiel OPAL-RT — position de SPÉCIFIER dans le cycle de vie.
- Contrat qualité officiel OPAL-RT — exigences et règles de qualité auxquelles les spécifications doivent contribuer.
- RACI officiel OPAL-RT — rôles et responsabilités associés à la définition, la validation et l'approbation des spécifications.
- Qualimétrie officielle OPAL-RT — vision officielle de la mesure de la qualité et de l'amélioration continue.
- Les métriques de la qualimétrie — référentiel opérationnel des KPI et signaux liés aux spécifications.
- Les tests — classification des spécifications et validations par niveau, interface, angle et type.
- Comment définir — transformation des besoins en spécifications exploitables.
- Exemples concrets — exemples de spécifications IF... THEN... par niveau et angle.
- Formation continue — parcours d'adoption des pratiques SDLC, des spécifications et des validations.